

6. Non-Functional Requirements

6.1 Scale assumptions. Tens of thousands of users with active plans; executions are monthly per plan but daily in aggregate. A “typical” day sees roughly `active_plans` / ~28 due plans, but common recurrence days (1st, 15th, month-end) likely see a multi-x spike — the Dispatch/Initiate fan-out (2.4) is sized for peak-day volume, not the average. API read/write traffic (plan CRUD, status polling) is assumed low-to-moderate, not the scaling bottleneck.

6.2 Scalability. Execution triggering scales horizontally by construction: the Dispatcher does a bounded paginated scan and the Initiator consumer group scales by adding instances up to the partition count (2.4). The read path scales independently via CQRS — Projection Workers and the Read Model scale without touching the write side. Kafka/database capacity is assumed platform-owned; this document covers application-level partitioning and consumer-group design only.

6.3 Reliability. Durability: zero event loss by construction (Outbox + Debezium, 2.2). Processing guarantee: at-least-once everywhere, made safe by idempotency (2.5) and a status-check-before-transition discipline (4.3), not by avoiding duplicates outright. Availability target (assumption pending confirmation, 5.2 #7): 99.9% for the HTTP API — a convenience surface, not trading-critical, since execution triggering is fully decoupled from API availability. RPO: effectively zero for committed writes. RTO: the pipeline self-heals from transient failures without manual intervention; the one case requiring it is the Dispatcher poison-pill (4.1), bounded by the alerting cutoff (assumption: same execution day).

6.4 Observability. Consumer lag on every Kafka consumer; Outbox backlog size and oldest-unpublished-row age (4.5); dispatch-completion alerting by a defined cutoff (4.1); order-command age/round-trip latency to Trading/Orders (4.2); correlation ids (`plan_id`, `execution_id`, (`execution_id`, `asset_id`)) threaded through every event, command, and log line; dashboards for daily dispatched-vs-processed count and an Execution status funnel with rejection rate by reason.

6.5 Security. Not designed in depth — this service reuses the App Backend’s existing Mobile-App authentication/authorization (out of scope per the brief) rather than introduce a new auth surface.